DHOLE PATIL EDUCATION SOCIETY'
DHOLE PATIL COLLEGE OF ENGINEERING
Accredited by NAAC with A+ Grade, An ISO 9001:2015 Certified Institute
1284, Near Eon IT Park Kharadi, Dhole Patil College Road, Wagholi, Pune-412207
Website: www.dpcoepune.edu.in E-mail:dpcoepune@gmail.com, Phone:020-66059900

EXPERIMENT NO: 01

TITLE: Study of Raspberry Pi, Arduino, and different operating systems for them.

AIM/ OBJECTIVES: Understanding the process of OS installation on Raspberry Pi/Arduino

REQUIREMENT:

SOFTWARE: Arduino IDE, Noobs OS
HARDWARE/EQUIPMENT: Arduino Board, Raspberry Pi Board

THEORY:

Arduino:

An Arduino board consists of an Atmel 8-bit AVR microcontroller with complementary components that facilitate programming and incorporation into other circuits.

It's an open-source physical computing platform based on a simple microcontroller board, and a development environment for writing software for the board.

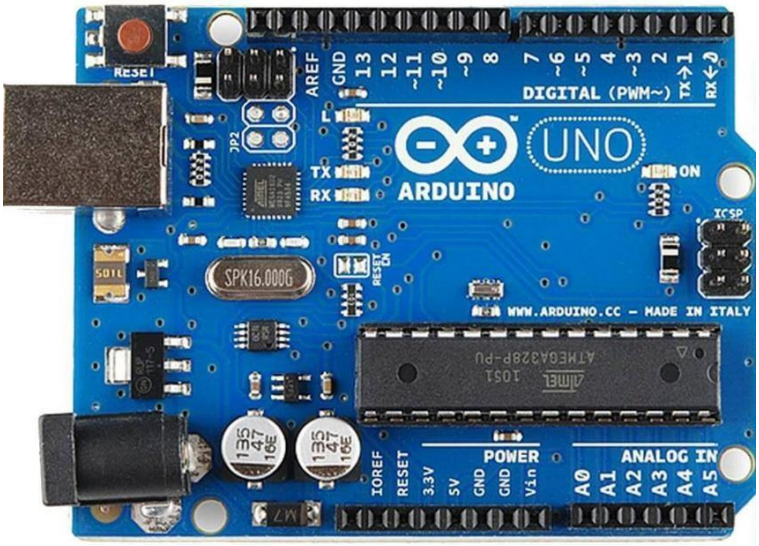


Fig.1 Arduino UNO

Specification of Arduino Uno:

- Microcontroller : ATmega328P
- Operating Voltage :5V
- Input Voltage (recommended) :7-12V
- Input Voltage (limit) :6-20V
- Digital I/O Pins :14 (of which 6 provide PWM output)
- Analog Input Pins :6
- DC Current per I/O Pin :20 mA
- DC Current for 3.3V Pin :50 mA
- Flash Memory :32 KB (ATmega328P) of which 0.5 KB used by bootloader
- SRAM :2 KB (ATmega328P)
- EEPROM :1 KB (ATmega328P)
- Clock Speed :16 MHz
- LED_BUILTIN :13

Raspberry Pi:

The Raspberry Pi is a series of small single-board computers developed in the United Kingdom by the Raspberry Pi Foundation to promote the teaching of basic computer science in schools and in developing countries.

The original model became far more popular than anticipated, selling outside of its target market for uses such as robotics.

Over 5 million Raspberry Pi is having been sold before February 2015, making it the best-selling British computer. By November 2016 they had sold 11 million units.



Fig.2 Raspberry Pi 3 B+

Operating Systems:

- The Foundation provides Raspbian, a Debian based Linux distribution for download, as well as third party Ubuntu, Windows 10 IOT Core, RISC OS, and specialized media center distributions.
- It promotes Python and Scratch as the main programming language, with support for many other languages.
- The default firmware is closed source, while an unofficial open source is available.



Fig.3 Symbol of Operating Systems

Peripherals:



Fig.4 Peripherals required for assembling



Fig.5 Setup of the Raspberry Pi

Procedure to install the Operating System:

- Install the SD Formatter software in Computer/Laptop
- Insert the micro SD card into the card reader and connect it to Computer/Laptop
- Select correct drive & format the SD card
- Download the OS from official web site of raspberry pi
- https://www.raspberrypi.org/downloads/noobs/
- Download zip file
- Extract the zip file into SD card
- Insert the micro SD card into the slot on the underside of the Pi
- Plug in keyboard and mouse
- Plug in monitor using the HDMI port
- The Raspberry Pi doesn't have a power button. It boots up as soon as you plug in the power supply

- If you've completed all the previous steps, plug in the power supply to boot the Raspberry Pi

CONCLUSION:

_____

_____

_____

Video Links: **Introduction to Arduino: https://youtu.be/XuKrmErjYmk**

Raspberry Pi OS: https://youtu.be/ntZToUT03dQ

EXPERIMENT NO: 02

TITLE: Open-source prototype platform- Raspberry-Pi/Beagle board/Arduino -Simple program digital read/write using LED and Switch -Analog read/write using sensor and actuators.

AIM/ OBJECTIVES: Understand the connection and configuration of General Purpose Input Output (GPIO) and its use in programming.

1 Write an application for the use of push switches and LEDs.

2 Write an application for Analog input and analog output

REQUIREMENT:

- **SOFTWARE: Arduino IDE**
- **HARDWARE/EQUIPMENT: Arduino Board, LED, Push Button**

THEORY: LED:

A light-emitting diode (LED) is a semiconductor light source that emits light when current flows through it. Electrons in the semiconductor recombine with holes, releasing energy in the form of photons. The color of the light (corresponding to the energy of the photons) is determined by the energy required for electrons to cross the band gap of the semiconductor. A push-button or simply button is a simple switch mechanism to control some aspect of a machine or a process. Buttons are typically made out of hard material, usually plastic or metal. The surface is usually flat or shaped to accommodate the human finger or hand, so as to be easily depressed or pushed. Buttons are most often biased switches, although many unbiased buttons (due to their physical nature) still require a spring to return to their un-pushed state. Terms for the "pushing" of a button include pressing depressing, mashing, slapping, hitting, and punching

PROCEDURE:

- Make the connection as per the circuit diagram
- Open Arduino IDE
- Select the Arduino Board
- Select the Arduino Port
- Write code in Editor Window and save the file
- Compile the code
- Upload the code

CIRCUIT/BLOCK DIAGRAM:

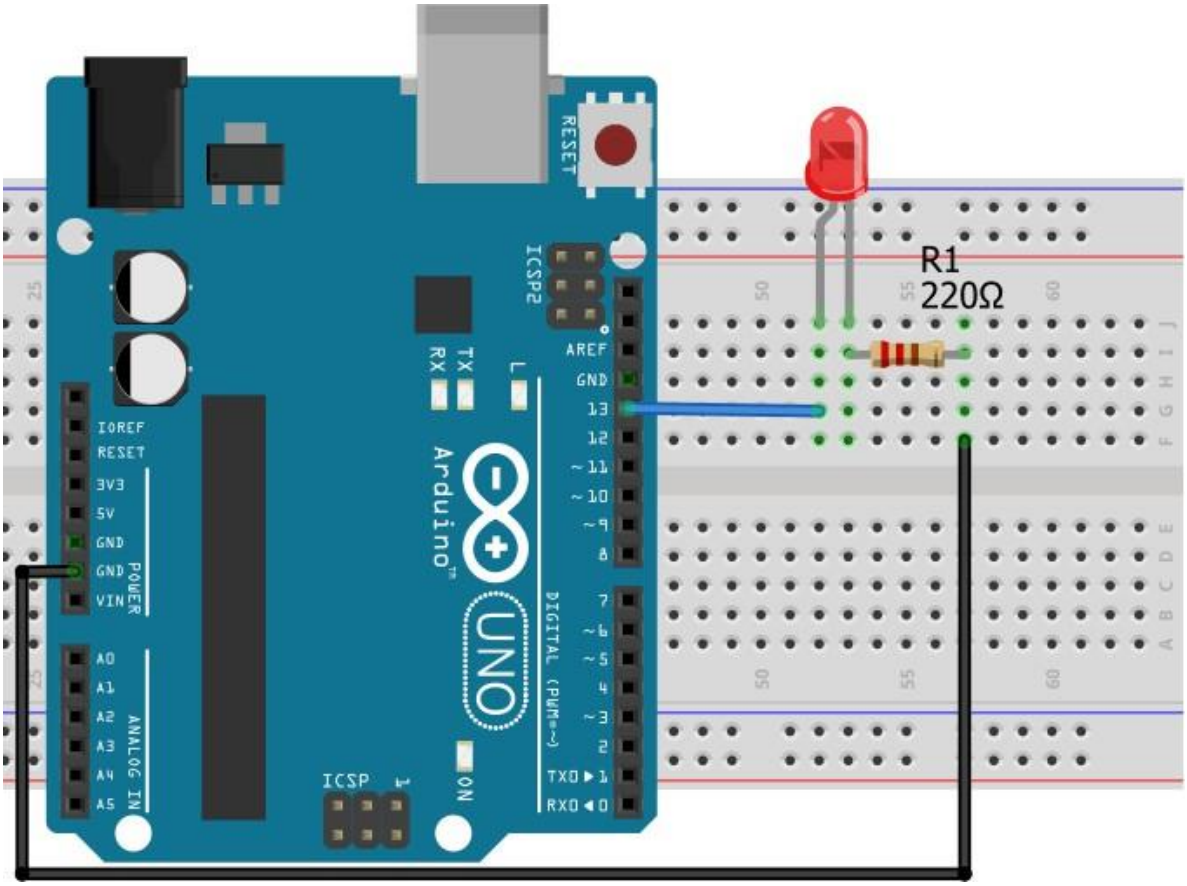


Fig.1 Interfacing of LED

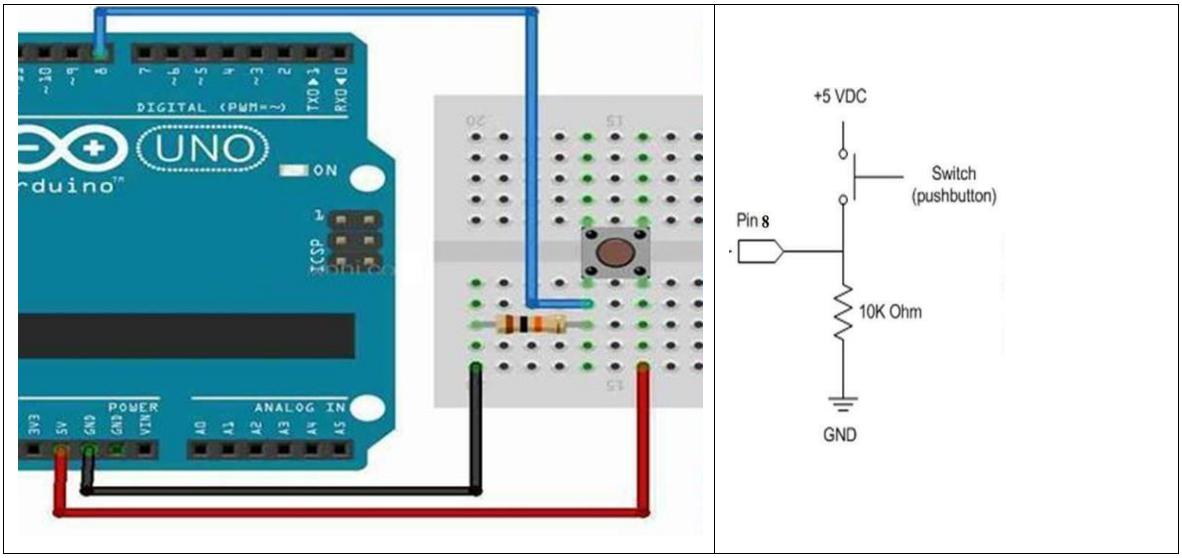


Fig.2 Interfacing of Push Button

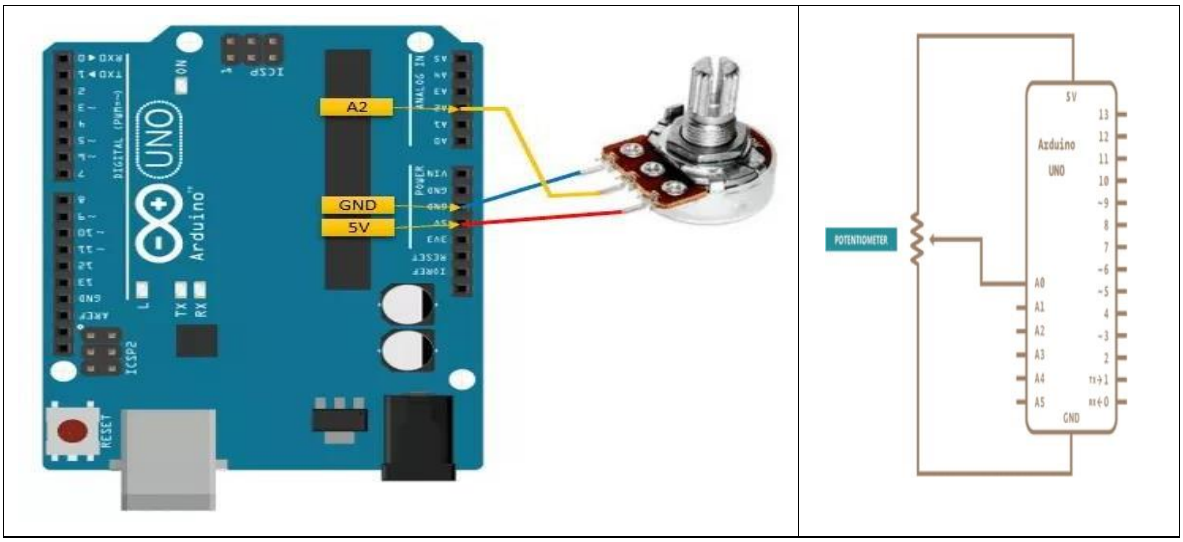


Fig3 Analog Input

PROGRAM CODE:

```
/* 1. Digital Read/Write*/

#define ledPin 13 // choose the pin for the LED
#define switchPin 8 // choose the input pin (for a pushbutton)

int val = 0; // variable for reading the pin status

void setup()
{
  pinMode(ledPin, OUTPUT); // declare LED as output
  pinMode(switchPin, INPUT); // declare pushbutton as input
}

void loop()
{
  val = digitalRead(switchPin); // read input value
  if (val == HIGH)
  { // check if the input is HIGH (button released)
    digitalWrite(ledPin, LOW); // turn LED OFF
  }
  else {
    digitalWrite(ledPin, HIGH); // turn LED ON
        }
  }
```

```
/* 2. Analog Read
*/
/* Reads an analog input on pin 0, prints the result to the Serial Monitor.  Graphical representation is available using Serial Plotter (Tools > Serial Plotter menu).  Attach the center pin of a potentiometer to pin A0, and the outside pins to +5V and ground.*/
// the setup routine runs once when you press reset:
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
}
// the loop routine runs over and over again forever:
void loop() {
  // read the input on analog pin 0:
  int sensorValue = analogRead(A0);
  // print out the value you read:
  Serial.println(sensorValue);
  delay(100);  // delay in between reads for stability
}

/* 3. Analog Read/Write*/
int PWM_Output = 10;
void setup() {
pinMode(PWM_Output,OUTPUT);
}
void loop() {
int analogValue = analogRead(A0)
//for(int i=0;i <255; i++)
{
analogWrite(PWM_Output,analogValue);
//analogWrite(PWM_Output,i);
delay(100);
} }
```

OBSERVATIONS:

	INPUT(Pin7)	**OUTPUT(pin 13)**
Digital Read/Write	HIGH	LOW
	LOW	HIGH
	ANALOG INPUT	**OUTPUT Count/Voltage**
Analog Read/Write	0V	0 count / 0V
	3.3V	670 / 3.3V
	5.0V	1023 / 5.0V

CONCLUSION:

EXPERIMENT NO: 03

TITLE: Interfacing sensors and actuators with Arduino/Raspberry-pi.

AIM/ OBJECTIVES:

Study of different sensors: - Temperature sensor, IR sensor, Ultrasonic sensor

REQUIREMENT:

- **SOFTWARE: Arduino IDE**
- **HARDWARE/EQUIPMENT: Arduino Board, LM 35 Temp. Sensor, IR Sensor, Ultrasonic Sensor**

THEORY:

LM 35 Temperature Sensor: LM35 is a temperature measuring device having an analog output voltage proportional to the temperature. It provides output voltage in Centigrade (Celsius). It does not require any external calibration circuitry. The sensitivity of LM35 is 10 mV/degree Celsius. As temperature increases, output voltage also increases. E.g. 250 mV means 25℃. It is a 3-terminal sensor used to measure surrounding temperature ranging from -55 ℃ to 150 ℃. LM35 gives temperature output which is more precise than thermistor output.

CIRCUIT/BLOCK DIAGRAM:

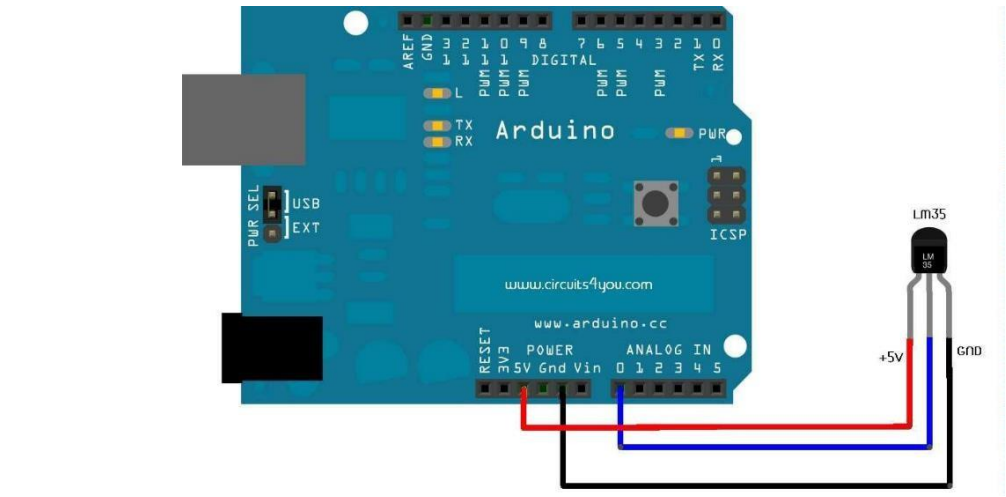


Fig.1 Interfacing of LM 35

IR Sensor:

IR sensor is an electronic device that emits the light in order to sense some object of the surroundings. An IR sensor can measure the heat of an object as well as detects the motion. Usually, in the infrared spectrum, all the objects radiate some form of thermal radiation. These types of radiations are invisible to our eyes, but infrared sensor can detect these radiations.

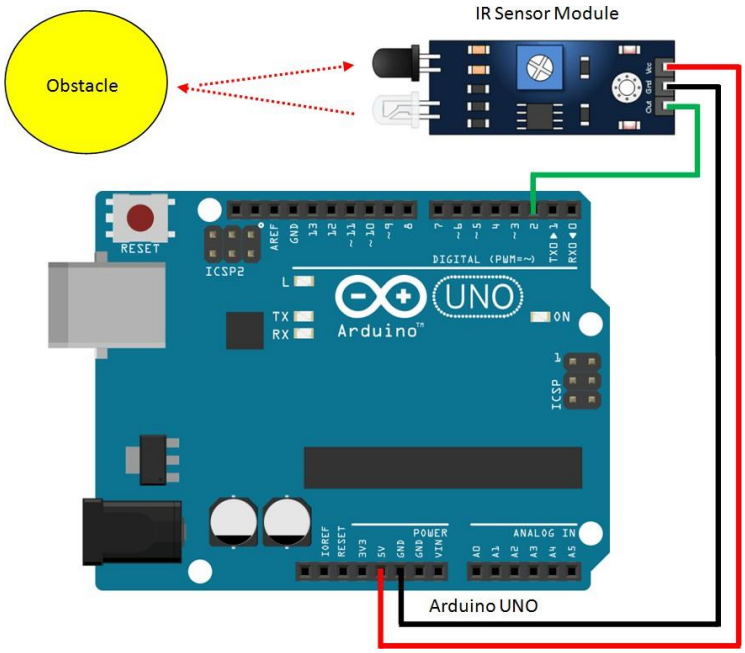


Fig.2 Interfacing of IR Sensor

Ultrasonic Sensor: Ultrasonic sensor is one of the main parts of numerous projects. Here we are going to interface a Ultrasonic sensor HC-SR 05 with Arduino Uno.

In this practical we discuss,

- Connect a Ultrasonic Sensor HC-SR 05 to Arduino.
- Read the sensor and convert it to length.
- Print the length to the Serial Monitor.
- Before starting we need to know about the working of Ultrasonic Sensor HC-SR 05. Actually it consists of an ultrasonic transmitter and an ultrasonic receiver. First ultrasonic transmitter sends an ultrasonic wave. This signal will collide with the object and reflect the signal. The receiver will receive the reflected signal. The distance calculated by the time taken to receive the reflected and the speed of sound in air. The speed of sound in air at room temperature is 340 Meter/Second or 0.034 centimeter/microsecond. The equation for calculating time is,

  Time=Distance/speed of sound

- If the object is 10 CM away from the sensor, You will get the time as per the equation is, 10/0.034 = 294.11 Microseconds
- But you will get the value from the Echo pin is 588.22. This is because of the sound wave needs to travel forward and bounce backward. So we need to divide that value by 2 to get the actual value (time). Here we want to calculate the distance from the time. So, re-arrange the equation we will get,

  Distance=Time x speed of sound

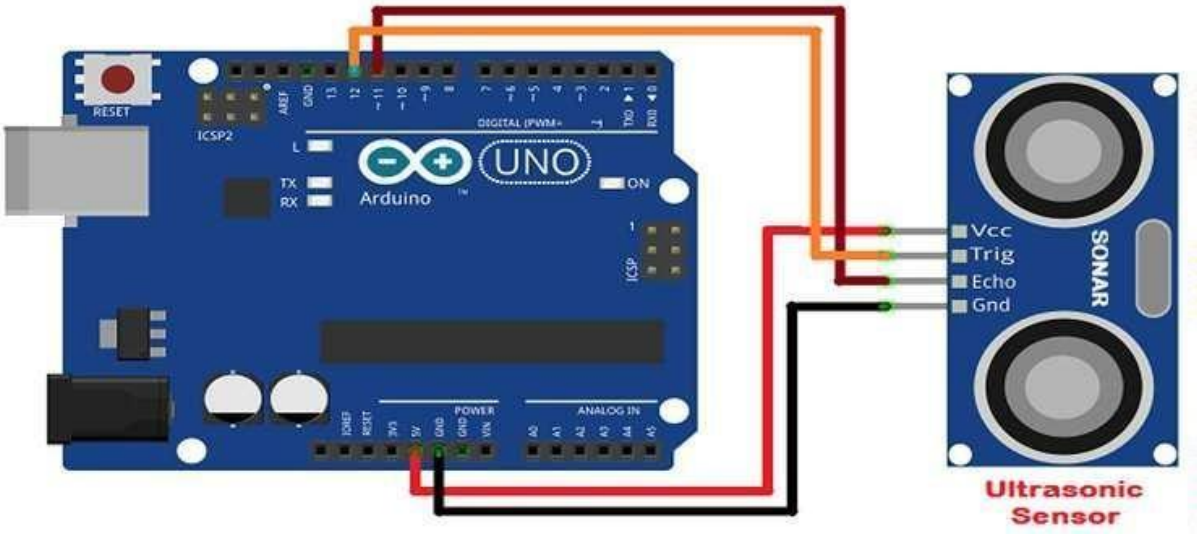


Fig.3 Interfacing of Ultrasonic Sensor

PROCEDURE:

- Make the connection as per circuit diagram
- Open Arduino IDE
- Select the Arduino Board
- Select the Arduino Port
- Write code in Editor Window and save file
- Compile the code
- Upload the code

PROGRAM CODE:

// 1. Interfacing of Temperature Sensor (LM35):

```
#define sensorPin A0
void setup() {
  // Begin serial communication at 9600 baud rate
  Serial.begin(9600);
}
void loop() {
  // Get the voltage reading from the LM35
  int reading = analogRead(sensorPin);

  // Convert that reading into voltage
  float voltage = reading * (5.0 / 1024.0);

  // Convert the voltage into the temperature in Celsius
  float temperatureC = voltage * 100;

  // Print the temperature in Celsius
  Serial.print("Temperature: ");
  Serial.print(temperatureC);
  Serial.print("\xC2\xB0"); // shows degree symbol
  Serial.print("C | ");

  // Print the temperature in Fahrenheit
  float temperatureF = (temperatureC * 9.0 / 5.0) + 32.0;
  Serial.print(temperatureF);
  Serial.print("\xC2\xB0"); // shows degree symbol
  Serial.println("F");
  delay(1000); // wait a second between readings
}
```

// 2. Interfacing of IR Sensor:

```
int ledPin=13;
int inputPin=2;
int val=0;
void setup()
{
 pinMode(ledPin,OUTPUT);
 pinMode( inputPin, INPUT);
 Serial.begin(9600);
}
void loop()
{
val=digitalRead(inputPin); // check the pin status (High=1/Low=0) //Active Low output
if(val==HIGH)
{
 Serial.print(“Object Absent\n");
 digitalWrite(ledPin,LOW);
 }
 else
 {
 Serial.print("Object Present\n");
 digitalWrite(ledPin,HIGH);
 }  }
```

/*3. Interfacing of Ultrasonic Sensor:

```
 Ultrasonic Sensor HC-SR04 interfacing with Arduino.*/
// defining the pins
const int trigPin = 9;
const int echoPin = 2;
// defining variables
long duration;
```

```
int distance;
Void setup() {
  pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
  pinMode(echoPin, INPUT); // Sets the echoPin as an Input
  Serial.begin(9600); // Starts the serial communication
  }
  Void loop () {
  // Clears the trigPin
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  // Sets the trigPin on HIGH state for 10 micro seconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  // Reads the echoPin, returns the sound wave travel time in microseconds
  duration = pulseIn(echoPin, HIGH);
  // Calculating the distance
  distance= duration*0.034/2;
  // Prints the distance on the Serial Monitor
  Serial.print("Distance: ");
  Serial.println(distance);
  }
```

OBSERVATIONS:

1. Interfacing of Temperature Sensor (LM35):

Temperature by Thermometer	**Sensor Reading**	**Temperature in ^{o}C** = (Sensor Reading) *100* (5.0 / 1024.0)	**Temperature in ^{o}F =**(temperature ^{o}C)* 9.0 / 5.0) + 32.0;
23	50	24	75
30	60	29	84
35	70	34	93

2. Interfacing of IR Sensor

Input Pin	Object Status
High	Object Absent
Low	Object Present

3. Interfacing of Ultrasonic Sensor

Distance (cm)	Duration (micro Seconds)	Distance (cm)= duration*0.034/2;
1	60	1.02
5	270	4.59
10	300	10.2

CONCLUSION:

_____

_____

_____

YouTube Video Link:

IR Sensor: https://youtu.be/OEmYIuwg2UA

PIR Sensor: https://youtu.be/K88t3uZSNdY

Ultrasonic Sensor: https://youtu.be/kQBnQK1Leac

Tinkercad Simulation Link:

https://www.tinkercad.com/dashboard?type=circuits&collection=designs

EXPERIMENT NO: 04

TITLE: IoT based Stepper Motor/DC Motor Control with Arduino/Raspberry Pi.

AIM/ OBJECTIVES: To study the DC Motor interfacing with Arduino

REQUIREMENT:

- **SOFTWARE: Arduino IDE**

- **HARDWARE/EQUIPMENT:** Arduino UNO board, PN2222 Transistor, Small 6V DC Motor, 1N4001 diode, 270 Ω Resistor

-

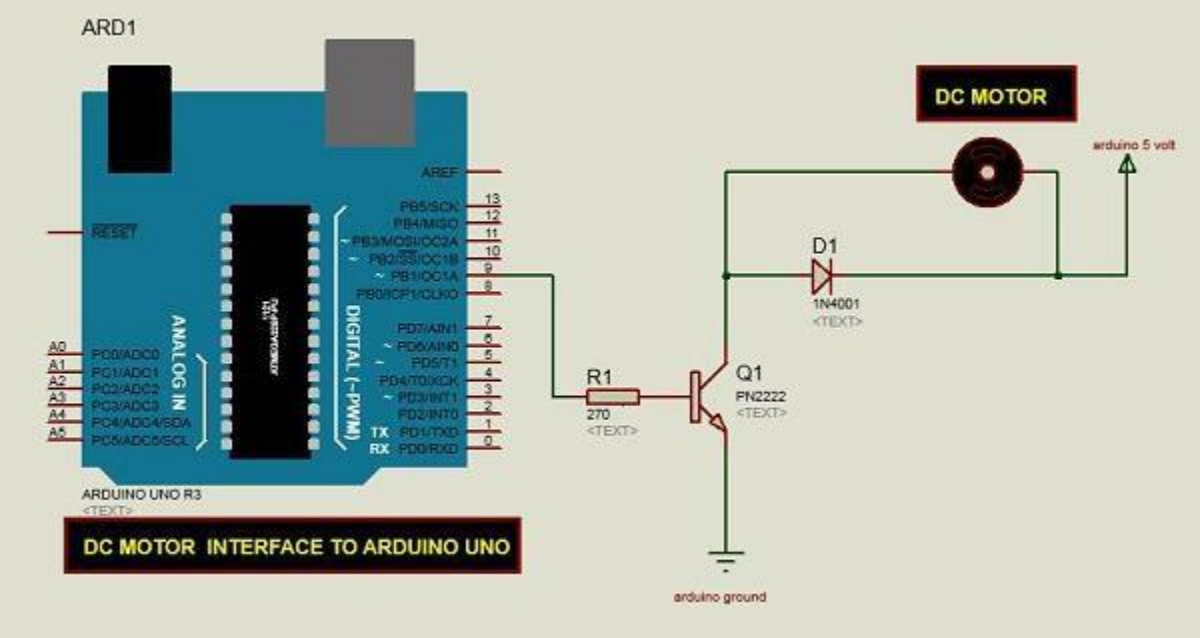
THEORY:

We will interface different types of motors with the Arduino board (UNO) and show how to connect the motor and drive it from the board.
There are three different types of motors −**DC motor, Servo motor and Stepper motor**

A DC motor (Direct Current motor) is the most common type of motor. DC motors normally have just two leads, one positive and one negative. If you connect these two leads directly to a battery, the motor will rotate. If you switch the leads, the motor will rotate in the opposite direction.

CIRCUIT/BLOCK DIAGRAM:

Warning − Do not drive the motor directly from Arduino board pins. This may damage the board. Use a driver Circuit or driver IC.



PROCEDURE:

Follow the circuit diagram and make the connections as shown in the figure.

The transistor acts like a switch, controlling the power to the motor.

Refer the following code. Arduino pin 3 is used to turn the transistor on and off and is given the name 'motorPin' in the sketch. Motor will spin in full speed when the Arduino pin number 3 goes high.

/* 1 Code to spin the motor*/

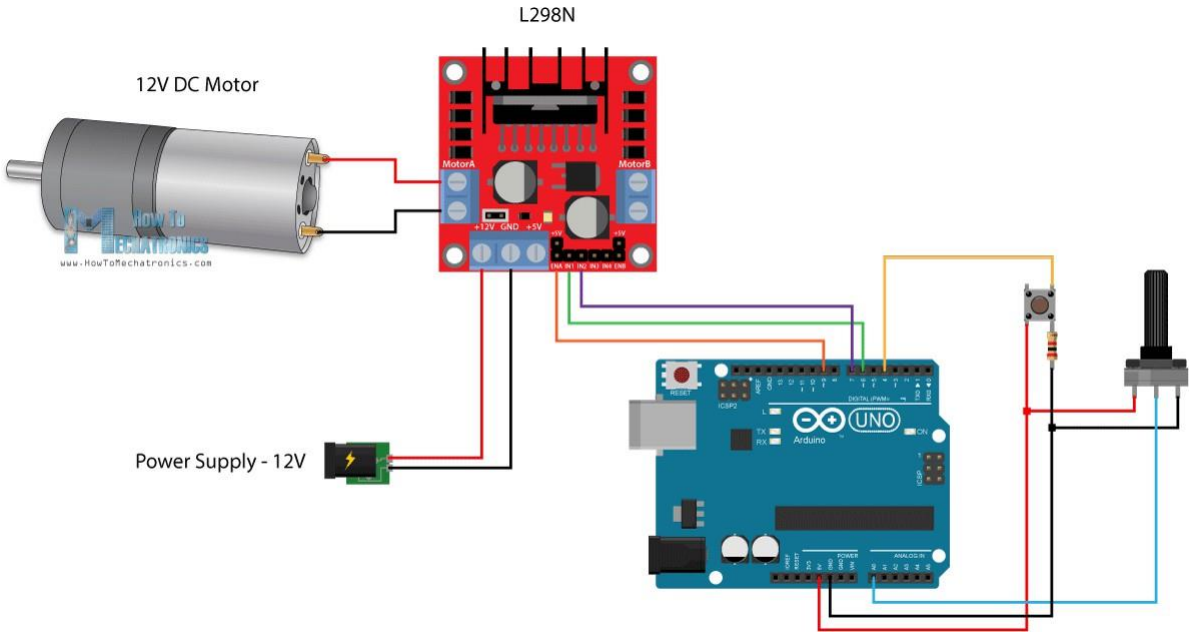
```
int motorPin = 3;
void setup() { }
void loop() {     digitalWrite(motorPin, HIGH);}
```

Instead of spinning at full speed, to control the DC motor speed, the code can be written as:

/*2 MOTOR SPEED CONTROL*/

```
int motorPin = 9;
void setup() {
  pinMode(motorPin, OUTPUT);
  Serial.begin(9600);
```

```
  while (! Serial);
  Serial.println("Speed 0 to 255");
}
void loop() {
  if (Serial.available()) {
    int speed = Serial.parseInt();
    if (speed >= 0 && speed <= 255) {
      analogWrite(motorPin, speed);  } }}
```




```
/* 3 DC Motor interfacing with Arduino through driver L298N
Speed Control of DC motor using PWM*/
int enable2 = 9;
int in3 = 10;
int in4 = 11;
//int analogValue = 200;  //= 100; //Any value      between 0 and 255

void setup() {
pinMode(enable2, OUTPUT);
pinMode(in3, OUTPUT);
pinMode(in4, OUTPUT);
```

```
}

void loop() {
//Clockwise rotation
//analogValue= analogRead(A0)  ;
for(int i=255;i > 0; i--)
analogWrite(enable2, i);
digitalWrite(in3, HIGH);
digitalWrite(in4, LOW);
//delay(10000);

//Anti-Clockwise rotation
//analogWrite(enable2, analogValue);
//digitalWrite(in3, LOW);
//digitalWrite(in4, HIGH);
//delay(10000);
}
```

CONCLUSION:

_____

_____

_____

EXPERIMENT NO: 05

TITLE: Introduction to MQTT/ CoAP and sending sensor data to cloud using Raspberry-Pi/Beagle board/Arduino

AIM/ OBJECTIVES: Interfacing of DHT11 (Humidity, Temperature) sensor and MQ3 Smoke Sensor with Raspberry Pi board and store the information on thingspeak cloud

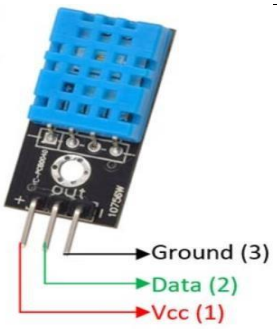
REQUIREMENT:

a) **SOFTWARE:** Python 3 IDE

b) **HARDWARE/EQUIPMENT:** Raspberry Pi 3 B+, DHT11 Sensor, MQ3 Smoke Sensor

THEORY:

DHT11 Sensor:

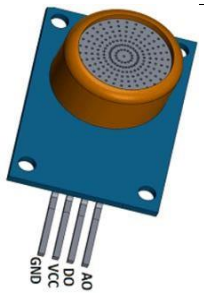
Specifications	Pin Configuration
• Low cost • 3 to 5V power and I/O • 2.5mA max current use during conversion (while requesting data) • Good for 20-80% humidity readings with 5% accuracy • Good for 0-50°C temperature readings ±2°C accuracy • Body size 15.5mm x 12mm x 5.5mm • 4 pins with 0.1" spacing	

The DHT11 is a basic, ultra low-cost digital temperature and humidity sensor. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air, and spits out a digital signal on the data pin (no analog input pins needed). It's fairly simple to use, but requires careful timing to grab data. The only real downside of this sensor is you can only get new data from it once every 2 seconds, so when using library, sensor readings can be up to 2 seconds old.

MQ3 Smoke Sensor:

The MQ3 Gas Sensor module detects gas leakage in home and industry. The MQ3 Gas Sensor can detect Alcohol, Benzine, Methane, Hexane, LPG, CO. It can detect the presence of alcohol gases at concentrations from 0.05 mg/L to 10 mg/L . The sensitive material used for this sensor is SnO2, whose conductivity is lower in clean air. Its conductivity increases as the concentration of alcohol gases increases. It has high sensitivity to alcohol and has a good resistance to disturbances due to smoke, vapor and gasoline.

This module provides both digital and analog outputs. MQ3 gas sensor module can be easily interfaced with Microcontrollers, Arduino Boards, Raspberry Pi etc. The MQ series of gas sensors use a small heater inside with an electrochemical sensor. They are sensitive to a range of gasses and are used indoors at room temperature. Due to its fast response time and high sensitivity, measurements can be taken as soon as possible. The sensor sensitivity can be adjusted by using the potentiometer.

Features:	**Pin out:**	
High sensitivity and fast response	Vcc – Power Supply (5V)	
Wide detecting scope	GND – Ground	
Stable and Long life	AO – Analog Output	
Simple drive circuit	DO – Digital Output	
Analog and Digital Outputs		

Raspberry pi 3:-

The Raspberry Pi is a low cost, credit-card sized computer that plugs into a computer monitor or TV, and uses a standard keyboard and mouse. It is a capable little device that enables people of all ages to explore computing, and to learn how to program in languages like Scratch and Python. It's capable of doing everything you'd expect a desktop computer to do, from browsing the internet and playing high-definition video, to making spreadsheets, word-processing, and playing games.

Raspberry Pi 3 Model B was released in February 2016 with a 64bit quad core processor, onboard Wi-Fi, Bluetooth and USB boot capabilities. On Pi Day 2018 model 3B+ appeared

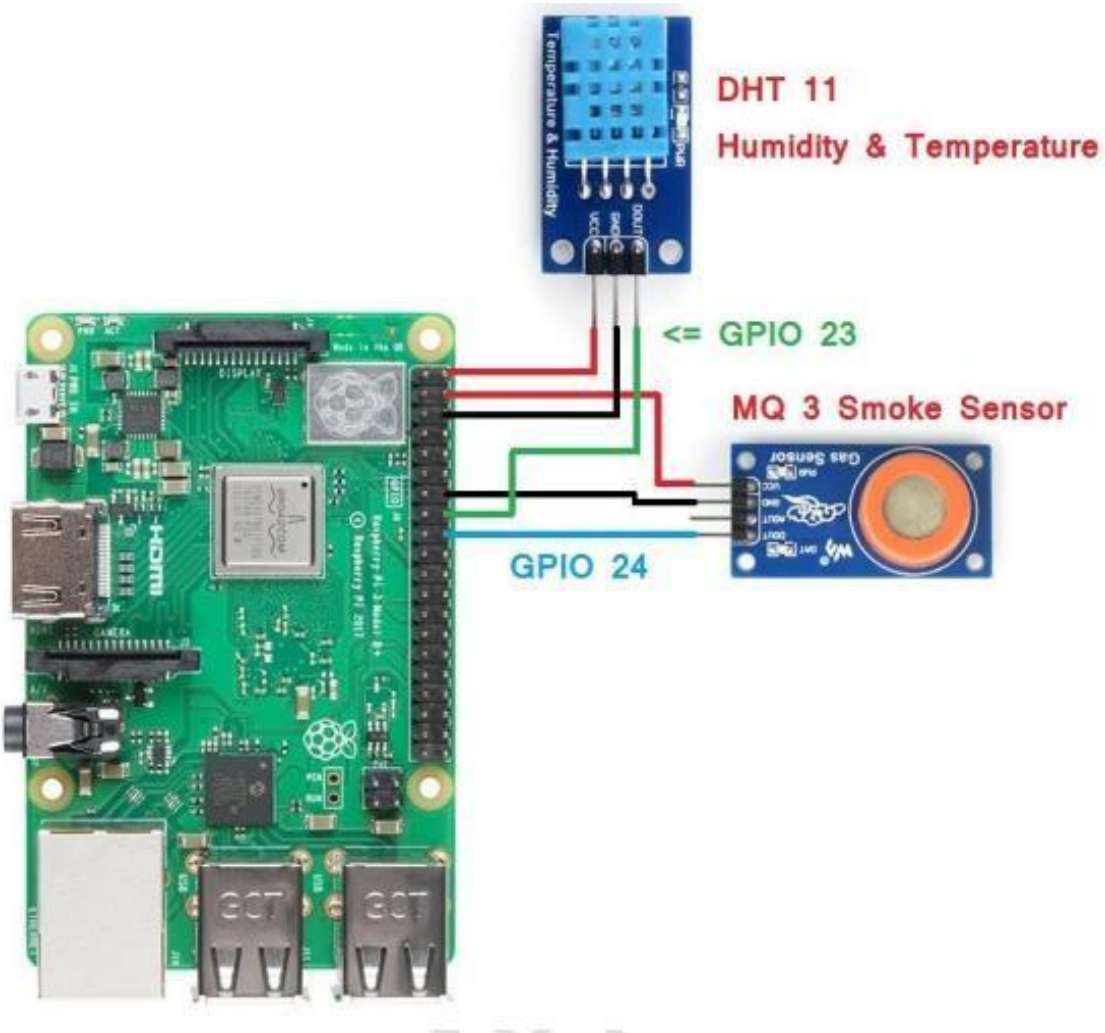
with a faster 1.4 GHz processor and a three times faster network based on gigabit Ethernet (300 Mbit / s) or 2.4 / 5 GHz dual-band Wi-Fi (100 Mbit / s). Other options are: Power over

Ethernet (PoE), USB boot and network boot (an SD card is no longer required). This allows the use of the Pi in hard-to-reach places (possibly without electricity).



Fig. Raspberry Pi 3 Model B+

CIRCUIT/BLOCK DIAGRAM:



PROCEDURE:

- Create a channel on the cloud (Thingspeak)
- Make the connection as per circuit diagram
- Open Python IDE
- Write code in Editor window and save file
- Compile the code
- Open thingspeak channel
- Observe the result

Program Code:

```
import sys
import Adafruit_DHT as dht
from time import sleep
import urllib
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
GPIO.setup(24, GPIO.IN) #Smoke Sensor Digital output
GPIO.setwarnings(False)
def DHT11_data():
 # Reading from DHT11 and storing the temperature and humidity
 humi, temp = dht.read_retry(11, 23)
 return humi, temp
# Enter Your API key here , below is sample key .  it will be different in your case.
myAPI = "ZQN4Z08XZ9HLOZY0"
# URL where we will send the data, don’t change it
baseURL = "https://api.thingspeak.com/update?api_key=%s" % myAPI
while True:
  Humidity, temperature = DHT11_data()
 print ('Temp: {0:0.1f} C Humidity: {1:0.1f} %'.
format(temperature,humidity))
 input_state = GPIO.input(24)
 if input_state == False:
 smoke = 1
```

```
print('Smoke Detected')
print(smoke)
else:
smoke = 0
print('Smoke is not Detected')
print(smoke)

from urllib.request import urlopen
content = urlopen(baseURL + "&field1=%s&field2=%s&field3=%s" %
(temperature,humidity,smoke))
sleep(15)
```

CONCLUSION:

_____

_____

_____

YouTube Video Link:

https://youtube.com/playlist?list=PLvmSxe_2n_6Ic56m7w_hmk8xSsQJAnDKX

Tinkercad Simulation Link:

https://www.tinkercad.com/dashboard?type=circuits&collection=designs

EXPERIMENT NO: 06

TITLE: Web based Home Automation

AIM/ OBJECTIVES: Get the status of a bulb at a remote place (on the LAN) through web.

REQUIREMENT:

SOFTWARE:

1. Arduino IDE

HARDWARE/EQUIPMENT:

1. NodeMCU
2. Connectors
3. Router
4. Relay Board

THEORY: SMART home technology use devices connected to the Internet of things (IoT) to automate and monitor in-home systems. SMART home technology allows users to control and monitor their connected home devices from SMART home apps, smart phones, or other networked devices. Users can remotely control connected home systems whether they are

home or away. This allows for more efficient energy and electric use as well as ensuring your home is secure. SMART home technology is now being used to create SMART cities.

Home automation or domestics is building automation for a home, called a smart home or smart house. A home automation system will control lighting, climate, entertainment systems, and appliances. It may also include home security such as access control and alarm systems. When connected with the Internet, home devices are an important constituent of the Internet of Things.

A home automation system typically connects controlled devices to a central hub or "gateway". The user interface for control of the system uses wall-mounted terminals, tablet or desktop computers, a mobile phone application, or a Web interface, that may also be accessible off-site through the Internet.

The practical aims at designing an advanced home automation system using normal web server and Wi-Fi technology. The devices can be switched ON/OFF and sensors can be read using a Personal Computer (PC) or mobile devices through Wi-Fi. Automation is the most frequently spelled term in the field of electronics. The hunger for automation brought many revolutions in the existing technologies. These had greater importance than any other technologies due to its user-friendly nature. These can be used as a replacement of the existing switches in home which produces sparks and also results in fire accidents in few situations. Considering the advantages of Wi-Fi an advanced automation system was developed to control the appliances in the house.

CIRCUIT/BLOCK DIAGRAM:

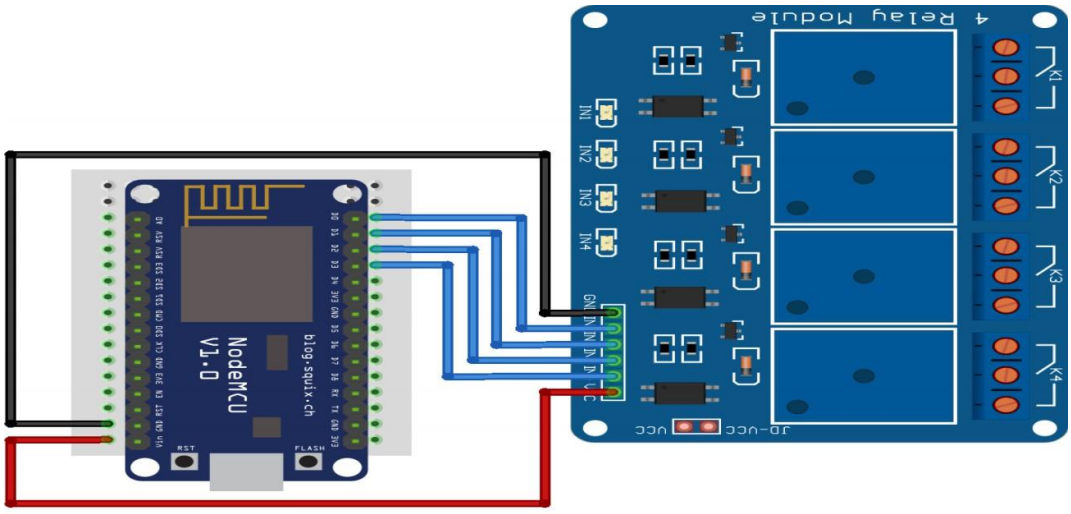


Fig1: Circuit diagram of NODEMCU and Relay Board

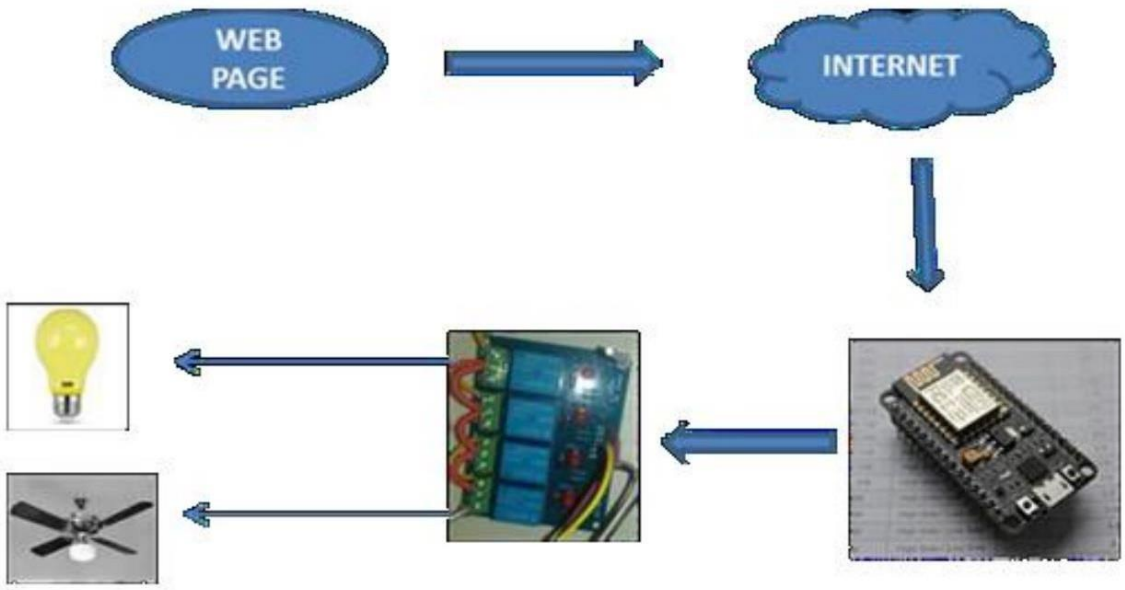


Fig2: Data Transfer between devices

Wi-Fi (Short for Wireless Fidelity) is a wireless technology that uses radio frequency to transmit data through the air. Wi-Fi has initial speeds of 1mbps to 2mbps. Wi-Fi transmits data in the frequency band of 2.4 GHz. It implements the concept of frequency division multiplexing technology. Range of Wi-Fi technology is 40-300 feet. The controlling device for the automation in the project is a Nodemcu. The data sent from PC over Wi-Fi will be received by Wi-Fi module connected to 9 Nodemcu. Nodemcu reads the data and decides the switching action of electrical devices connected to it through Relays.

Day by day, the field of automation is blooming and these systems are having great impact on human beings. The project which is to be implemented is a home automation using Easy IOT Webserver and WIFI and has very good future development. In the current system webserver is installed on a windows PC so the home appliances can be controlled using only by using the device on which webserver is installed. This can be further developed installing webserver on cloud.

Advantage of installing web server on the cloud is that home can be controlled by using any device which has WIFI 802.1 and a web browser. By visiting the IP address of the cloud the control actions can be taken.

PROCEDURE:

1. Connect Nodemcu mobile on the same network.
2. Compile and download the code to the Nodemcu.
3. Connect relay board to Nodemcu and connect appliances to the relay board.
4. Open any browser in mobile and connect to the IP provide by the Nodemcu on the

Serial terminal (Ex. 192.168.123.19).

5. Login with credentials and turn ON/OFF device with Buttons.

OBSERVATIONS:

CONCLUSION:

YouTube Video Link:

https://youtube.com/playlist?list=PLvmSxe_2n_6Ic56m7w_hmk8xSsQJAnDKX

Tinkercad Simulation Link:

https://www.tinkercad.com/dashboard?type=circuits&collection=designs

EXPERIMENT NO: 07

TITLE: Interfacing Arduino to Bluetooth Module

AIM/ OBJECTIVES: To interface the HC-06 Bluetooth module with arduino. To learn about the module - the basic introduction. To learn how to set up the module, connect and interface it.

REQUIREMENT:

SOFTWARE: Arduino IDE

HARDWARE/EQUIPMENT:

Arduino Nano - You could use any other development board if you wish
HC-06 Bluetooth module
Resistors - 1kΩ and 2kΩ
Male-to-Male Jumper wires
Male-to-Female Jumper wires

THEORY:

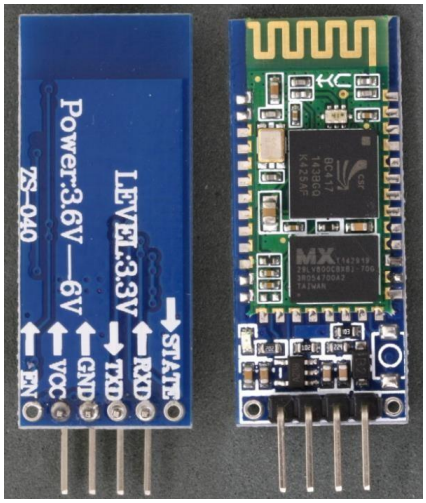
HC-06 Bluetooth module:

HC-06 is a Bluetooth module designed for establishing short range wireless data communication (<100 meters). It is very easy to interface and communicate. It can be interfaced with almost all microcontrollers or processors as it uses UART interface.

This module has the ability to transmit files at speed up to 2.1Mbps and works on Bluetooth 2.0 communication protocol. Unlike the HC-05 Bluetooth module, this module can only act as a slave device.

- Operating Voltage: 3.3V - 6V
- Operating Frequency range: 2.402 GHz - 2.480 GHz

Pin out



You will only need the four pins in the HC-06 Bluetooth module.

- RXD: Serial Data Receive Pin. Used for serial input. 3.3V logic
- TXD: Serial Data Transmit Pin. Used for serial output. 3.3V logic
- GND: Ground
- VCC: +5V

Setting up the HC-06 Bluetooth module

To set up your HC-06 Bluetooth module, you will need the USB-TTL Serial Converter module. Connect your USB-TTL Serial converter module with your Bluetooth module. The connections are as follows:

- RXD ---> TX

- TXD ---> RX
- GND ---> GND
- VCC ---> 5V

Now, plug in the USB-TTL Serial converter module to your Laptop/ PC. The LED on the converter module should light up if it's working properly. The HC-06 Bluetooth module is permanently configured to be slave and is always in AT mode when not paired to any other device.

Open Arduino IDE and go to Tools, then Serial Monitor.

Set the Baud rate to 9600 and the 'line ending' to 'Both NL and CR'.

Type in 'AT' without the quotes and click send. If the Serial Monitor displays "OK", the module is in AT mode and you are good to go.

Use these AT commands for the corresponding functions:

- AT : Check the connection
- AT+NAME: To check the default name of the module
- AT+VERSION: To obtain the version
- AT+BAUD: To check the default baud rate of the Bluetooth module
- AT+PIN: To obtain the password of the module
- AT+NAME'DESIRED NAME': Changes the default name of the module to the DESIRED NAME

To change the baud rate of the HC-06 Bluetooth module, type in AT+BAUDx, where x has range from 1 - 9, inclusive.

- 1: 1200 baud
- 2: 2400 baud
- 3: 4800 baud
- 4: 9600 baud (default)
- 5: 19200 baud
- 6: 38400 baud
- 7: 57600 baud
- 8: 115200 baud

Sending AT+BAUD3 will change the baud rate from 9600 baud to 4800 baud.

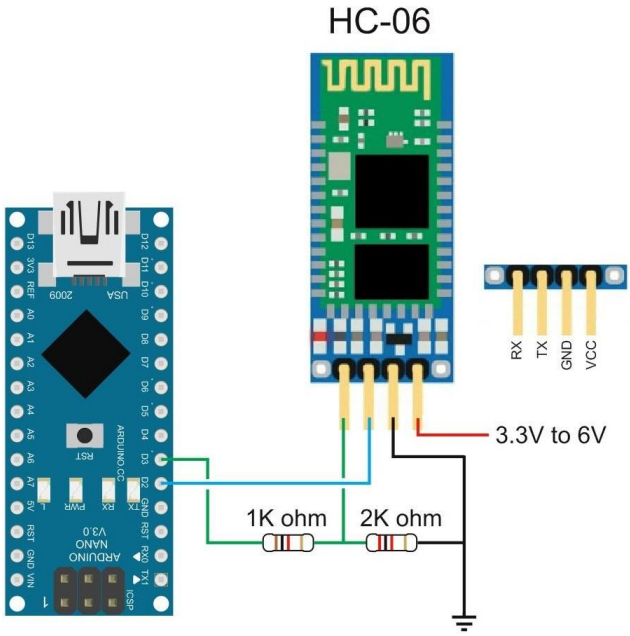
Refer to the schematic to see how the Bluetooth module should be connected with the Arduino development Board. Connecting the HC-06 Bluetooth module with Arduino development

board Hardware components Arduino Nano - You could use any other development board if you wishHC-06 Bluetooth module Resistors - 1kΩ and 2kΩ (I didn't have a 2kΩ resistor with me so I used 2 1kΩ resistors instead)Male-to-Male Jumper wires Male-to-Female Jumper wires Software Arduino IDE. The Arduino boards have a serial port at D0 and D1 and it communicates with these pins as well as the computer via USB. We are using the Serial functions so we cannot use these pins. Therefore, we will be using the D2 and D3 pins for communication with the HC-06 Bluetooth module. D2 pin will be connected to TXD pin of the HC-06 module whereas the D3 pin will be connected to the RXD pin.

The module is connected to +5V standard regulated power supply and the voltage divider is used for converting the 5V logic signal sent by Arduino D3 pin to 3.3V logic signals which is suitable for the module. If you do not understand the voltage divider principle, please refer to the YouTube video in the last section of this post.

If separate power sources are used for the Bluetooth module and Arduino development board, their ground must be connected for voltage reference.

CIRCUIT/BLOCK DIAGRAM:



The Bluetooth module the ZS-040 is based on, the EGBT-046S, is a 3.3V device. The HC-06 breakout board has a 3.3v regulator that allows a larger input voltage to be used, in the range of 3.6 to 6 volts. The RX pin can still only accept 3.3V though. This means a voltage divider is required to connect to a 5V Arduino. A simple voltage divider can be created using 2 resistors. I am using a 1K ohm resistor and a 2K ohm resistor. The Arduino will read 3.3V as a HIGH so the HC-06 TX pin can be connected directly to the Arduino.

Test Communication With The HC-06

After connecting everything we need to talk to the HC-06. We can do this by using software serial on the Arduino. I use software serial to talk to Bluetooth modules and use the hardware serial for debugging.

The following sketch takes whatever is entered in to the serial monitor on a host computer and relays it to the HC-06. The sketch also takes whatever the HC-06 outputs and forwards it to the serial monitor. The Arduino is acting like a relay station between the serial monitor and the BT module.

The HC-06s I have have a default baud rate of 9600. Other modules have a different baud rate. If 9600 doesn't work try other speeds. 38400 is also very common. Once you have communication working you can change the baud rate to suit your needs.

PROGRAM CODE:

```
// Basic Bluetooth sketch HC-06_01
// Connect the Hc-06 module and communicate using the serial monitor
// The HC-06 defaults to AT mode when first powered on.
// The default baud rate is 9600
// The Hc-06 requires all AT commands to be in uppercase. NL+CR should not be added to the
command string

#include <SoftwareSerial.h>
SoftwareSerial BTserial(2, 3); // RX | TX
// Connect the HC-06 TX to the Arduino RX on pin 2.
// Connect the HC-06 RX to the Arduino TX on pin 3 through a voltage divider.


void setup()
{
    Serial.begin(9600);
    Serial.println("Enter AT commands:");

    // HC-06 default serial speed is 9600
    BTserial.begin(9600);
}
```

```
void loop()
{

    // Keep reading from HC-06 and send to Arduino Serial Monitor

    if (BTserial.available())
    {
        Serial.write(BTserial.read());
    }

    // Keep reading from Arduino Serial Monitor and send to HC-06
    if (Serial.available())
    {
        BTserial.write(Serial.read());    }}
```

Compile and upload the sketch to Arduino development board.

Testing the Bluetooth Communication

Now that you have successfully set up the Bluetooth module, let's test the communication between this module and another device. If you have an Android device, such as phone or tablet, go to Play Store and install 'Bluetooth Terminal' app from Qwerty.

Make sure you have turned on Bluetooth on your device.

Open the menu icon on the top right corner of the screen and tap on 'Connect a device - secure'. As we have not paired the Bluetooth module with the device, tap on 'Scan for devices'. You will see the Name of your Bluetooth module. To pair with your module, the device will ask for the password. After you provide the correct password, you will see that the device and the module are connected. The LED on the Bluetooth module will flash every 2 seconds.

Open the Serial monitor in the Arduino IDE and set the correct baud rate. Send a message from the Bluetooth terminal app and you will see the message displayed on the Serial monitor. Try sending a message from the Serial monitor. If the connection is successful, the message will be received by your Android device.

OBSERVATIONS:

CONCLUSION:

_____

_____

_____

Link: https://create.arduino.cc/projecthub/RucksikaaR/interfacing-the-hc-06-bluetooth-module-with-arduino-f9c315

EXPERIMENT NO: 08

TITLE: IoT based small project on a topics like Smart cities, chat bot, smart home (Home Automation), and environmental issues e.t.c.

AIM/ OBJECTIVES: Implement a smart home automation system. The system automates home appliances and controls them over internet from anywhere.

REQUIREMENT:

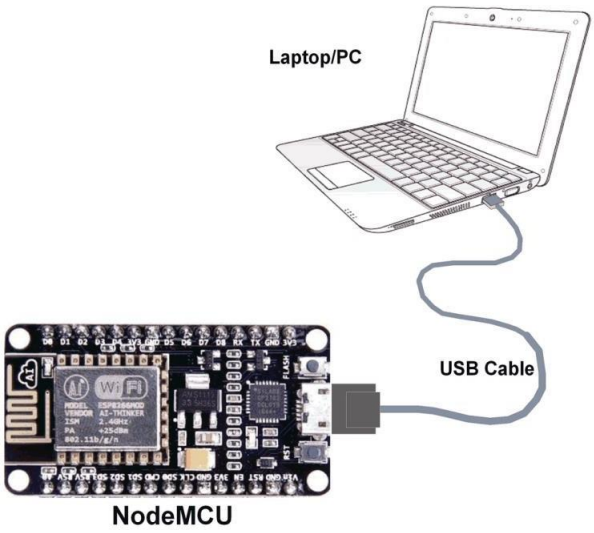
SOFTWARE: Arduino IDE, Google Firebase, Android App

HARDWARE/EQUIPMENT: NodeMCU, Relay, AC Bulb

THEORY:

NodeMCU: NodeMCU is Lua based firmware of ESP8266. Generally, ESPlorer IDE is referred for writing Lua scripts for NodeMCU. It requires getting familiar with ESPlorer IDE and Lua scripting language.

There is another way of developing NodeMCU with a well-known IDE i.e. Arduino IDE. We can also develop NodeMCU applications using the Arduino development environment. This makes things easy for Arduino developers than learning a new language and IDE for NodeMCU.



Relay: A relay is an electrically operated switch. It consists of a set of input terminals for a single or multiple control signals, and a set of operating contact terminals. The switch may have any number of contacts in multiple contact forms, such as make contacts, break contacts, or combinations thereof.



CIRCUIT/BLOCK DIAGRAM:

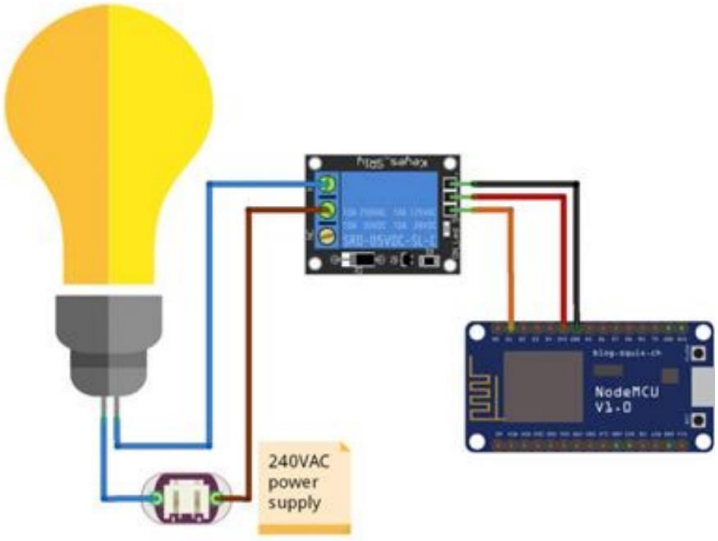


Fig.1 Circuit diagram of Home Automation System

PROCEDURE:

- Make the connection as per circuit diagram
- Open Arduino IDE
- Select the NodeMCU ESP8266
- Select the communication Port
- Write code in Editor Window and save file
- Compile the code
- Upload the code
- Open the Android app
- Control the appliances

OBSERVATIONS:

CONCLUSION:

_____

_____

_____

YouTube Video Link:

https://youtube.com/playlist?list=PLvmSxe_2n_6Ic56m7w_hmk8xSsQJAnDKX

Tinkercad Simulation Link:

https://www.tinkercad.com/dashboard?type=circuits&collection=designs

EXPERIMENT NO: 09

TITLE: Communicate between Arduino and Raspberry PI using any wireless medium like ZigBee

AIM/ OBJECTIVES:

REQUIREMENT:

a) **SOFTWARE:**

c) **HARDWARE/EQUIPMENT:**

b) **THEORY:**

CIRCUIT/BLOCK DIAGRAM: OBSERVATIONS:

CONCLUSION: